

NeRF · 零基础工程报告

ESE 650 HW3 Problem 3 (LEGO 数据集)

Student ID: 18330723

2026 年 4 月 25 日

阅读说明：本报告假设你会 **PyTorch 基础** (**forward, backward, Adam**)，但没听过 **NeRF**、**体渲染**、**positional encoding**。我会从一堆照片 JPG 和 JSON 开始讲起，一路推到能生成新视角图。每个工程决策都会问“为什么”。

目录

1 要解决的真实问题	2
1.1 本题的具体设定	2
2 先看数据：JSON 和图片里有什么	3
2.1 文件夹结构	3
2.2 JSON 文件解析	3
2.3 相机 pose 是什么？	3
2.4 相机内参 (intrinsics): focal length	4
3 3D 用什么表示？为什么选神经场？	5
3.1 候选方案对比	5
3.2 神经场是什么	5
3.3 为什么 MLP 连续可表示 3D？	5
4 从 3D 场到 2D 像素：体渲染	6
4.1 问题：NN 给了 $(\mathbf{c}, \sigma)$ ，怎么画出一张图？	6
4.2 物理起源：放射传输方程	6
4.3 每一项的直观意义	6
4.4 离散化： α -compositing	6
4.5 $\sum T_k \alpha_k \leq 1$ ：漂亮的恒等式	7
4.6 代码实现	7
5 从像素到光线：ray generation	8
5.1 任务：每个像素一条光线	8
5.2 相机坐标系下的光线方向	8

5.3	变到世界坐标系	8
5.4	代码实现	8
6	沿光线采样: stratified sampling	10
6.1	为什么不能均匀采样?	10
6.2	Stratified Sampling 的做法	10
6.3	代码	10
7	喂给 MLP 之前: Positional Encoding	11
7.1	神经网络的“低频偏好”	11
7.2	Positional Encoding: 把低频变高频	11
7.3	为什么有效?	11
7.4	代码	11
7.5	L 选多少?	12
8	TinyNeRF 网络架构	13
8.1	代码	13
8.2	为什么只有 32 dim、4 层就够?	13
8.3	为什么 σ 用 ReLU、 $\mathbf{c}$ 用 sigmoid?	13
9	完整训练 Pipeline	14
9.1	一次 iteration 的步骤	14
9.2	代码	14
9.3	关键超参数	14
9.4	为什么整张图一起训 (batch = 整图)?	15
10	训练结果验证	16
10.1	三种验证方式	16
10.2	Novel View 的实现	16
11	常考陷阱 (速查)	17
12	考题: 按学习路径排序	18

1 要解决的真实问题

想象你在逛博物馆

你走进博物馆，看到一只漂亮的 LEGO 小推土机。你围着它拍了 100 张照片，从不同角度：正面、侧面、俯视、仰视...

回家后你想：能不能从一个我没拍过的角度看它？比如“从正上方往下看”或“从它肚子底下往上看”？

如果可以，你就“拥有”了这只 LEGO ——以任意视角重现它，哪怕原本没拍那个角度。

为什么这很难

你只有 100 张离散的 **2D** 图片，而要生成的是任意视角的 **2D** 图。中间必须有某种**3D 表示**——否则你怎么知道从没拍过的角度那只推土机长啥样？

问题是：这个 **3D 表示**是什么？怎么从 **2D** 图自动学出来？

1.1 本题的具体设定

数据：100 张 800×800 的 LEGO 多视角渲染图 + 每张的相机位姿 + 相机内参。

简化：为了在 CPU 上能训练，resize 到 100×100 。

要产出：

- 一个模型（神经网络的权重），能接受任意相机 pose 输出对应的 2D 图。
- 训练 loss 曲线、训练集对比、novel view（训练 pose 之间的插值）。

2 先看数据：JSON 和图片里有什么

2.1 文件夹结构

NeRF LEGO 数据布局

```
data/
  transforms_colmap.json  # 相机参数 + 每张图的 pose
  images/
    train/
      r_0.png             # 800x800 LEGO 渲染图
      r_1.png
      ... (100 张)
```

2.2 JSON 文件解析

transforms_colmap.json 长这样：

JSON 结构（节选）

```
{
  "camera_angle_x": 0.6911,          // 水平视场角（弧度）
  "frames": [
    {
      "file_path": "./train/r_0",
      "transform_matrix": [           // 4x4 相机到世界矩阵
        [r11, r12, r13, tx],
        [r21, r22, r23, ty],
        [r31, r32, r33, tz],
        [0, 0, 0, 1]
      ]
    },
    ... (共 100 帧)
  ]
}
```

2.3 相机 pose 是什么？

camera-to-world 变换矩阵（extrinsics）

一个 4×4 齐次矩阵 $T_w^c = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$ ，把点从相机坐标系变到世界坐标系：

$$\mathbf{p}_{\text{world}} = T_w^c \cdot [\mathbf{p}_{\text{camera}}; 1]$$

- $R \in SO(3)$ ：相机的朝向（从世界看相机 orientation 怎么摆）。
- $\mathbf{t} \in \mathbb{R}^3$ ：相机的位置（在世界坐标系中）。

所以 transform_matrix[:3, 3] 就是相机的 3D 位置；transform_matrix[:3, :3] 是它的旋转矩阵。

2.4 相机内参 (intrinsics): focal length

`camera_angle_x` 是水平视场角 (FOV), 要转成 focal length f (像素单位):

从 FOV 推 focal length

对水平方向, FOV θ_x 和图像宽度 W 、focal length f 的关系:

$$\tan(\theta_x/2) = \frac{W/2}{f} \implies f = \frac{W/2}{\tan(\theta_x/2)}$$

本题 focal length 计算

```
orig_W = 800
focal_orig = 0.5 * orig_W / np.tan(0.5 * camera_angle_x)    # 约 1111 px
# resize 到 100x100 后, focal 按比例缩放
focal = focal_orig * (100.0 / 800)                          # = 138.89 px
```

为什么 focal 要跟着 `resize` 变? 因为 focal 和图像分辨率是同一维度 (像素); 图像变小, focal 也要按比例缩小才能保持物理含义一致.

3 3D 用什么表示？为什么选神经场？

3.1 候选方案对比

表示	怎么存	问题
Mesh	顶点 + 三角面	离散、不可微、难优化；难表达半透明、毛发
Voxel Grid	$N \times N \times N$ 体素，每格存颜色 + 密度	内存 $O(N^3)$ 爆炸； $N = 512$ 就 $\sim 128M$
Point Cloud	一堆 3D 点	无表面、渲染需要额外算法、颗粒感
Neural Field	一个神经网络 $F_\theta(\mathbf{x}) \rightarrow (\mathbf{c}, \sigma)$	连续、参数少、可微，但慢

3.2 神经场是什么

Neural Field（神经场）

把 3D 空间表示成一个函数：

$$F_\theta : \mathbb{R}^3 \rightarrow \mathbb{R}^4, \quad F_\theta(\mathbf{x}) = (\mathbf{c}, \sigma)$$

- 输入：任意 3D 位置 $\mathbf{x} \in \mathbb{R}^3$.
- 输出：颜色 $\mathbf{c} \in [0, 1]^3$ 和体密度 $\sigma \in \mathbb{R}^+$.

“函数”由 MLP 实现，权重就是 θ 。学 3D 场景 = 学 θ 。

物理含义：

- $\sigma(\mathbf{x})$ ：位置 $\mathbf{x}$ 处的体密度（单位 1/长度）。值大 $\rightarrow$ 这里有东西（不透明）；值 0 $\rightarrow$ 这里是空气。
- $\mathbf{c}(\mathbf{x})$ ：位置 $\mathbf{x}$ 处的颜色（假设自发光，不考虑光照）。

3.3 为什么 MLP 连续可表示 3D？

一个 4 层 MLP 是个**参数化的连续函数**，输入 (x, y, z) 任意点都能输出值。所以它天然就是连续表示，不像 voxel 有离散格子。

“用网络压缩场景”的妙处

本题 TinyNeRF 只有 4292 个参数 $\rightarrow$ 用 4KB 左右存下了一只 LEGO 的 3D 表示！比对应 voxel grid 小几个量级。

4 从 3D 场到 2D 像素：体渲染

4.1 问题：NN 给了 $(\mathbf{c}, \sigma)$ ，怎么画出一张图？

每个像素对应一条光线（从相机出发穿过像素射向 3D 场景）。我们需要沿这条光线累积它穿过的所有 $(\mathbf{c}, \sigma)$ ，得到该像素的颜色。

4.2 物理起源：放射传输方程

从光传输说起

一束光沿 t 方向走，它的亮度 $L(t)$ 同时经历两件事：

1. 被吸收： $\frac{dL}{dt} = -\sigma(t)L(t)$ （每米损失 σ 比例的光）。
2. 被发射： $\frac{dL}{dt} = +\sigma(t)c(t)$ （这一段介质自发光贡献）。

合起来： $\frac{dL}{dt} = -\sigma L + \sigma c$ 。

解这个 ODE（积分因子法）得**体渲染方程**：

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t)) dt$$

其中**透射率**

$$T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$

4.3 每一项的直观意义

三个量的物理直觉

- $\sigma(\mathbf{x}) dt$ ：光在位置 $\mathbf{x}$ 被吸收的概率密度。
- $T(t)$ ：光从起点 t_n 穿到 t 还没被吸收的概率。
- $T(t)\sigma(t) dt$ ：光恰好在 $[t, t + dt]$ 被吸收的概率。

体渲染 = 沿光线每个位置被吸收的概率 $\times$ 那里的颜色，积分出总颜色。

4.4 离散化： α -compositing

真实世界我们不能算无穷积分，只能沿光线**采样** N 个点 $\{t_k\}_{k=1}^N$ ，每段 $[t_k, t_{k+1}]$ 假设 σ 常数。

记 $\delta_k = t_{k+1} - t_k$ ，段内积分：

$$\int_{t_k}^{t_{k+1}} T \sigma_k \mathbf{c}_k dt = T_k \mathbf{c}_k \int_{t_k}^{t_{k+1}} e^{-\sigma_k(s-t_k)} \sigma_k ds = T_k \mathbf{c}_k (1 - e^{-\sigma_k \delta_k})$$

定义**段 opacity** $\alpha_k = 1 - e^{-\sigma_k \delta_k}$ ，则所有段求和：

$$\hat{C} = \sum_{k=1}^N T_k \alpha_k \mathbf{c}_k, \quad T_k = \prod_{j=1}^{k-1} (1 - \alpha_j)$$

4.5 $\sum T_k \alpha_k \leq 1$: 漂亮的恒等式

累加恒等式

注意 $T_k \alpha_k = T_k - T_{k+1}$ (因为 $T_{k+1} = T_k(1 - \alpha_k)$), 所以

$$\sum_{k=1}^N T_k \alpha_k = \sum_{k=1}^N (T_k - T_{k+1}) = T_1 - T_{N+1} = 1 - T_{N+1}$$

剩余的 $T_{N+1} \in [0, 1]$ 就是背景的权重 (光穿过整条光线还没被吸收的概率)。本题背景默认黑, 贡献 0; 有时会设成白色或灰色。

4.6 代码实现

volume_rendering

```
def volume_rendering(radiance_field, ray_origins, depth_values):
    # radiance_field: (H, W, N, 4), 最后一维 [r, g, b, sigma]
    sigma = F.relu(radiance_field[..., 3]) # 非负
    rgb = torch.sigmoid(radiance_field[..., :3]) # [0, 1]

    # 段间距 delta_k = t_{k+1} - t_k
    dists = depth_values[..., 1:] - depth_values[..., :-1]
    dists = torch.cat([dists, 1e10 * torch.ones_like(dists[..., :1])], -1)

    # alpha_k = 1 - exp(-sigma * delta)
    alpha = 1.0 - torch.exp(-sigma * dists)

    # T_k = cumprod(1 - alpha), 前置 1
    ones = torch.ones_like(alpha[..., :1])
    T = torch.cumprod(torch.cat([ones, 1.0 - alpha + 1e-10], -1), -1)[..., :-1]

    weights = T * alpha # (H, W, N)
    rgb_map = torch.sum(weights[..., None] * rgb, dim=-2) # (H, W, 3)
    return rgb_map
```

为什么 δ_N 设成 10^{10} ?

最后一段代表“从最远采样点到无穷远”。把 δ_N 设极大, 让 $\alpha_N = 1 - e^{-\sigma_N \cdot 10^{10}} \approx 1$ (只要 $\sigma_N > 0$), 保证光线被完全“吸收”, 背景不透进来。

另一个说法: 这让 $T_{N+1} \approx 0$, 剩余权重 0, $\sum T_k \alpha_k = 1$ 。

5 从像素到光线: ray generation

5.1 任务: 每个像素一条光线

对 100×100 图像的每个像素 (i, j) , 构造一条 3D 光线 $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ (起点 $\mathbf{o}$, 方向 $\mathbf{d}$) .

5.2 相机坐标系下的光线方向

针孔相机模型

相机在原点, 沿 $-Z$ 看过去。像素 (i, j) 对应相机坐标系的方向:

$$\mathbf{d}_{\text{cam}} = \left(\frac{j - W/2}{f}, -\frac{i - H/2}{f}, -1 \right)$$

约定: $+X$ 右、 $+Y$ 上、 $+Z$ 向后 (所以相机看 $-Z$) .

为什么 y 分量是 $-(i - H/2)$? 因为图像像素 (i, j) 中 i 是行号 (从上往下增), 而相机 $+Y$ 朝上; 所以要翻转符号.

5.3 变到世界坐标系

方向 $\mathbf{d}$ 需要绕相机到世界的旋转矩阵 R :

$$\mathbf{d} = R \cdot \mathbf{d}_{\text{cam}}, \quad \mathbf{o} = T_w^c[:3, 3] \text{ (相机位置)}$$

$\mathbf{o}$ 对所有像素都一样 (同一张图所有光线同一起点 = 相机位置) .

5.4 代码实现

get_rays

```
def get_rays(H, W, f, trans_matrix):
    # 构造 (H, W) 的像素索引网格
    i = torch.arange(H, dtype=torch.float32)
    j = torch.arange(W, dtype=torch.float32)
    ii, jj = torch.meshgrid(i, j, indexing='ij')    # 都是 (H, W)

    # 相机坐标系方向
    directions_cam = torch.stack([
        (jj - W*0.5) / f,          # x
        -(ii - H*0.5) / f,        # y (翻转)
        -torch.ones_like(ii)      # z (朝 -Z)
    ], dim=-1)                      # (H, W, 3)

    # 旋转到世界坐标系
    R = trans_matrix[:3, :3]
    ray_directions = (directions_cam[..., None, :] * R).sum(dim=-1)
    ray_directions = ray_directions / ray_directions.norm(dim=-1, keepdim=True)

    # 光线起点 = 相机位置 (对所有像素相同)
    ray_origins = trans_matrix[:3, 3].expand_as(ray_directions)
```

```
return ray_origins, ray_directions
```

6 沿光线采样: stratified sampling

6.1 为什么不能均匀采样?

想象对每条光线都在 $[t_n, t_f]$ 均匀取 $N = 32$ 个点:

$$t_1 = t_n + \frac{1}{32}(t_f - t_n), t_2 = t_n + \frac{2}{32}(\dots), \dots$$

均匀采样的坑

这些采样点对所有训练 iteration 都**完全一样**。NN 只需要“在这 32 个位置”记住正确的 $(\mathbf{c}, \sigma)$ 就能拟合训练图 $\rightarrow$ 学到**表式函数**而不是连续场 $\rightarrow$ novel view 完全失败。

6.2 Stratified Sampling 的做法

分层采样

把 $[t_n, t_f]$ 分成 N 个等长 bin。在第 k 个 bin 内**随机**取一个点:

$$t_k = t_n + \frac{k-1}{N}(t_f - t_n) + \xi_k, \quad \xi_k \sim \mathcal{U}(0, \frac{t_f - t_n}{N})$$

每次 iteration 抖动 ξ_k 不同 $\rightarrow$ NN 被迫学到**连续函数** (因为采样点哪里都可能出现)。

6.3 代码

sample_points_from_rays

```
def sample_points_from_rays(ray_origins, ray_directions, near, far, N):
    # 均匀的  $t_k$ 
    t_vals = torch.linspace(0.0, 1.0, N)                # (N,)
    depth_values = near + t_vals * (far - near)          # (N,)

    # 每个 bin 内抖动
    bin_width = (far - near) / N
    noise = torch.rand(H, W, N) * bin_width
    depth_values = depth_values[None, None, :] + noise  # (H, W, N)

    # 3D 采样点 =  $\mathbf{o} + t * \mathbf{d}$ 
    sampled_points = ray_origins[..., None, :] + depth_values[..., :, None]
    * ray_directions[..., None, :]
    return sampled_points, depth_values                  # (H, W, N, 3), (H, W, N)
```

本题设置: $N = 32$ 个采样点, $t_n = 2.0$, $t_f = 6.0$.

为什么 **2.0–6.0**? 因为 LEGO 在场景原点附近, 相机距离大约 4 左右; $[2, 6]$ 能 cover 整个物体同时不浪费采样在空区。

7 喂给 MLP 之前: Positional Encoding

7.1 神经网络的“低频偏好”

直接把 (x, y, z) 喂给 MLP, 会发现: 学出来的图糊成一团。没有纹理细节、边缘模糊, 颜色区分不开。

Spectral Bias (频谱偏好)

MLP 倾向于拟合低频函数 $\rightarrow$ 用 ReLU 或 tanh 激活的 MLP 很难学习“近处颜色剧烈变化”这类高频信号。

理论来源: Neural Tangent Kernel 分析表明 MLP 的 eigenvalue 在低频大、高频小 $\rightarrow$ 低频特征被优先拟合。

7.2 Positional Encoding: 把低频变高频

Positional Encoding (Fourier features)

$$\gamma(\mathbf{x}) = (\mathbf{x}, \sin 2^0 \pi \mathbf{x}, \cos 2^0 \pi \mathbf{x}, \sin 2^1 \pi \mathbf{x}, \cos 2^1 \pi \mathbf{x}, \dots, \sin 2^{L-1} \pi \mathbf{x}, \cos 2^{L-1} \pi \mathbf{x})$$

本题 $L = 10$ (省略 π 用 2^k)。维度 $3 \cdot (1 + 2L) = 3 \cdot 21 = 63$ 。

对每个坐标分量独立做: $x \in \mathbb{R}$ 被映到 21 维, 3 个坐标合起来 63 维。

7.3 为什么有效?

NTK 视角的直觉

PE 把 MLP 的 NTK 从低通变成 band-limited (覆盖 2^{L-1} 以内的所有频率) $\rightarrow$ MLP 能学到这个频率带以下的所有细节。

换句话说: 把学“高频 $f(x)$ ”的问题 $\rightarrow$ 转化为学“低频 $g(\sin 2^{L-1} x)$ ”, 后者 MLP 擅长。

7.4 代码

positional_encoding

```
def positional_encoding(x, num_frequencies=10, include_input=True):
    # x: (N, 3)
    encodings = []
    if include_input:
        encodings.append(x)
    for k in range(num_frequencies):
        freq = 2.0 ** k
        encodings.append(torch.sin(freq * x))
        encodings.append(torch.cos(freq * x))
    return torch.cat(encodings, dim=-1)  # (N, 63)
```

7.5 L 选多少?

L 取值	效果	问题
$L = 2$	糊、低频主导	丢高频细节
$L = 10$	本题默认	够用于 100×100 图
$L = 20$	高频过度	训练慢、易过拟合、“checkerboard” artifact

8 TinyNeRF 网络架构

MLP 架构

层	形状	激活
Input PE	63	-
Linear 1	63 $\rightarrow$ 32	ReLU
Linear 2	32 $\rightarrow$ 32	ReLU
Linear 3	32 $\rightarrow$ 32	ReLU
Linear 4	32 $\rightarrow$ 4	无 (split)
Output[3]	σ	ReLU
Output[: 3]	$\mathbf{c}$	sigmoid

总参数: $(63 + 1) \cdot 32 + (32 + 1) \cdot 32 + (32 + 1) \cdot 32 + (32 + 1) \cdot 4 = 4292$.

8.1 代码

TinyNeRF

```
class TinyNeRF(nn.Module):
    def __init__(self, pos_dim=63, fc_dim=32):
        super().__init__()
        self.nerf = nn.Sequential(
            nn.Linear(pos_dim, fc_dim), nn.ReLU(),
            nn.Linear(fc_dim, fc_dim), nn.ReLU(),
            nn.Linear(fc_dim, fc_dim), nn.ReLU(),
            nn.Linear(fc_dim, 4)          # 输出 (r, g, b, sigma)
        )
    def forward(self, x):
        return self.nerf(x)
```

8.2 为什么只有 32 dim、4 层就够？

- LEGO 结构规整（直角、平面多），不是毛发等复杂几何。
- 图像分辨率低（ 100×100 ） $\rightarrow$ 细节需求低。
- PE 把难度转移到输入空间，MLP 本身不用太大。

真正的 NeRF 是 8 层 $\times$ 256 hidden $\rightarrow \sim 500K$ 参数，能处理 800×800 复杂场景。

8.3 为什么 σ 用 ReLU、 $\mathbf{c}$ 用 sigmoid？

- σ 必须非负（物理量：密度，不能为负） $\rightarrow$ ReLU（也可用 softplus 更平滑）。
- $\mathbf{c} \in [0, 1]^3$ （归一化颜色） $\rightarrow$ sigmoid 自然约束到该区间。

9 完整训练 Pipeline

9.1 一次 iteration 的步骤

1. 随机选 1 张训练图 I_i 和它的 pose T_i .
2. `get_rays`: 对每个像素发光线 $\rightarrow (\mathbf{o}, \mathbf{d})$ 形状 $(H, W, 3)$.
3. `sample_points_from_rays`: 沿光线分层采样 $\rightarrow (H, W, N, 3)$ 空间点.
4. Flatten $(H \cdot W \cdot N, 3)$, 过 positional encoding $\rightarrow (H \cdot W \cdot N, 63)$.
5. Mini-batch 过 MLP (每次 16384 点, 防爆显存) $\rightarrow (H \cdot W \cdot N, 4)$.
6. Reshape 回 $(H, W, N, 4)$, 分离 σ 和 $\mathbf{c}$.
7. `volume_rendering`: α -composite 每条光线 $\rightarrow (H, W, 3)$ 预测图.
8. MSE loss: $\|\hat{I} - I_i\|_2^2$.
9. Backprop, Adam 更新 θ .

9.2 代码

主训练循环

```
optimizer = torch.optim.Adam(model.parameters(), lr=5e-3)
torch.manual_seed(9458); np.random.seed(9458)

for it in tqdm(range(1000)):
    train_idx = np.random.randint(n_train)
    img = images[train_idx]; pose = poses[train_idx]

    rgb_pred = nerf_step_forward(H, W, focal, pose,
                                near=2.0, far=6.0, num_samples=32,
                                get_minibatches, model)

    loss = F.mse_loss(rgb_pred, img)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    losses.append(loss.item())
```

9.3 关键超参数

超参	值
Near / Far	2.0 / 6.0
N_{sample}	32
L (PE frequencies)	10
Hidden dim	32
Layers	4
LR	5×10^{-3} (Adam)
Iterations	1000
Image size	100×100
Seed	9458

9.4 为什么整张图一起训 (batch = 整图)?

- 图小 ($100 \times 100 \times 32$ 采样点 $\approx 320K$ 点) $\rightarrow$ 显存勉强放得下.
- 每次迭代看到完整的图 $\rightarrow$ loss 信号强、收敛快.

真正的 NeRF 处理 800×800 时只能 **sample rays** ——每次迭代随机采 1024 条光线 (而不是整图), 否则显存爆.

10 训练结果验证

10.1 三种验证方式

1. **Loss 曲线**: MSE 应单调下降 (前期快、后期平)。本题从 ~ 0.3 降到 ~ 0.02 (图像像素值 $\in [0, 1]$) .
2. **训练集重建**: 渲染一张训练图片, 和 GT 对比。应该视觉接近, 细节可能略糊.
3. **Novel view (训练 pose 插值)**: 选两张训练 pose T_A, T_B , 插值出 5 个中间 pose, 渲染 $\rightarrow$ 合理过渡.

10.2 Novel View 的实现

简单线性插值 (本题用法)

```
for idx in range(5):
    t = idx / 4.0                # t in [0, 1]
    pose_novel = (1 - t) * pose_A + t * pose_B    # lerp
    rgb = render(pose_novel, ...)
```

Pose 线性插值是错的 (但本题能凑合)

$T = (1 - t)T_A + tT_B$ 对平移是对的 (线性空间), 但对旋转矩阵错的:

- 两个 $R \in SO(3)$ 的线性组合不再正交 ($R_{\text{mid}}^\top R_{\text{mid}} \neq I$) .
- 严格做法: R 用 SLERP (quaternion) 或 $SO(3)$ 上 exp-log 插值; t 用 lerp.

本题插值距离近、图小 $\rightarrow$ 偏差视觉上不明显, 能凑合看.

11 常考陷阱（速查）

Trap 1 · “ σ 是什么？为什么过 ReLU？”

σ 是体密度（不是像素 opacity），单位 1/长度，必须非负 $\rightarrow$ ReLU.

常混淆：“像素级 α ” 是 $\alpha_k = 1 - e^{-\sigma_k \delta_k}$ ，才是 $[0, 1]$ 的 opacity.

Trap 2 · “如果不做 PE 会怎样？”

MLP 学出来的图会非常模糊——纹理、边缘、高频细节全丢掉，像整体平滑过。可以做消融实验对比： $L = 0$ vs $L = 10$.

Trap 3 · “ δ_N 为什么那么大？”

10^{10} 代表“到无穷远”。 $\alpha_N = 1 - e^{-\sigma_N \cdot 10^{10}} \approx 1$ （只要 $\sigma_N > 0$ ） $\rightarrow$ 保证背景不透进来.

Trap 4 · “训练只看到 100 张图，为什么能生成新视角？”

隐式 3D 一致性：要同时解释 100 张图， F_θ 学到的必须是一个一致的 3D 结构（否则不同视图会打架）。从这个 3D 结构渲染新视角自然合理.

这就是 NeRF 的 “monitor consistency $\Rightarrow$ 3D structure” 的魅力.

Trap 5 · “TinyNeRF 只有 4292 参数为啥能学 LEGO？”

三点：

- PE 把难度转移到输入，MLP 不用大.
- LEGO 结构规整，不是毛发/云雾.
- 图像 100×100 ，细节需求低.

如果原图是 800×800 带纹理，4292 参数肯定不够.

Trap 6 · “本题简化 $F_\theta(\mathbf{x})$ 不看 view direction $\mathbf{d}$ ，问题？”

同一个 3D 点从不同角度颜色相同 $\rightarrow$ 无法建模镜面反射/高光（金属、水面）。本题 LEGO 是漫反射主导，无所谓；真 NeRF 用 $F_\theta(\mathbf{x}, \mathbf{d})$.

Trap 7 · “如果相机位姿不准会怎样？”

NeRF 严重依赖 pose 精度。位姿差 1° 就会让图像明显糊（有研究工作 BARF、Nerf- 联合优化 pose 和 F_θ ）.

本题 COLMAP 位姿比较精确，所以效果 OK.

12 考题：按学习路径排序

数据与问题理解（必会）：

1. 输入给了什么？输出要什么？
2. JSON 里的 `transform_matrix` 是什么坐标变换？
3. `camera_angle_x` 怎么换算 focal length? `resize` 后怎么处理？
4. 为什么用神经场而不是 mesh/voxel/point cloud？

体渲染（核心）：

5. 写出体渲染方程，解释 σ, T, c 各自的物理意义。
6. 从连续方程推 $\alpha_k = 1 - e^{-\sigma_k \delta_k}$ 。
7. 证明 $\sum_k T_k \alpha_k \leq 1$ 。
8. 剩余的 T_{N+1} 代表什么？本题什么取值？
9. 为什么 σ 过 ReLU、 c 过 sigmoid？互换会怎样？
10. δ_N 为什么要设 10^{10} ？

光线生成与采样（中级）：

11. `get_rays` 里为什么 y 要乘 -1 ？
12. 光线方向变到世界坐标系只用 R 不用 t ，为什么？
13. Stratified sampling 相比均匀采样的好处？
14. 若不 stratified，NN 会学到什么？

Positional Encoding（重点）：

15. 为什么 PE 有必要？（spectral bias）
16. $L = 10$ 输入维度怎么算？
17. L 太小 / 太大各有什么问题？
18. PE 的本质是什么？（Fourier features / NTK band-limiting）

架构与训练（中级）：

19. TinyNeRF 为啥只有 4292 参数还能学？
20. 为什么本题能 `batch = 整张图`？真 NeRF 为啥要 `sample rays`？
21. Loss 为什么是 MSE？L1 / LPIPS 会不会更好？
22. `get_minibatches` 为什么要切成 16384 点一块？

验证与失败模式（加分）：

23. Novel view 用 pose lerp 为什么错？正确做法？
24. 如果训练 pose 有 1° 误差，NeRF 会怎样？

- 25. 为什么只有 100 张图也能学出 3D? “隐式一致性” 什么意思?
- 26. 本题简化 $F_{\theta}(\mathbf{x})$ 不用视角 $\mathbf{d}$, 哪些场景会失败?
- 27. 如何把本题扩展到 full NeRF? 代价和收益?

“3D 场 $\xrightarrow{\text{体渲染}}$ 2D 图 $\xrightarrow{MSE}$ 梯度 $\rightarrow$ 更新场。
NeRF 的全部就是这个循环。”